

A Guide to Programming Intel IA32 PC Architecture

Kai Li, Princeton University
First draft, 1999
Revised 2003

[1 Intel IA32 Processors](#)

[1.1 Modes](#)

[1.2 Register Set](#)

[1.3 Addressing](#)

[1.4 Processor Reset](#)

[2 Assembly Programming](#)

[2.1 Instruction Syntax](#)

[2.2 Memory Operands](#)

[2.3 Frequently Used Instructions](#)

[2.4 Assembler Directives](#)

[2.5 Inline Assembly](#)

[2.6 Program Structure and Calling Convention](#)

[3 BIOS Services](#)

[3.1 Display Memory](#)

[3.2 Write to Display at Current Cursor](#)

[3.3 Read from Diskette](#)

The goal of this documentation is to provide a brief description of the Intel IA32 PC architecture, a brief introduction to assembly programming using the Gnu assembler, and a small set of BIOS services that can be used in the course projects.

References:

- *IA32 Intel Architecture Software Developer's Manual, Volume 2: Instruction Set Reference Manual*, Intel Corporation, 2003
- *IA32 Intel Architecture Software Developer's Manual, Volume 3: Operating System Writer's Manual*, Intel Corporation, 2003.
- http://www.gnu.org/manual/gas-2.9.1/html_mono/as.html
- *The Undocumented PC: A Programmer's Guide to I/O, CPUs, and Fixed Memory Areas, 2nd Edition*, Frank van Gilluwe, Addison-Wesley Developers Press, 1997.

1 Intel IA32 Processor

Intel uses IA32 to refer to Pentium processor family, in order to distinguish them from their 64-bit architectures.

1.1 Modes

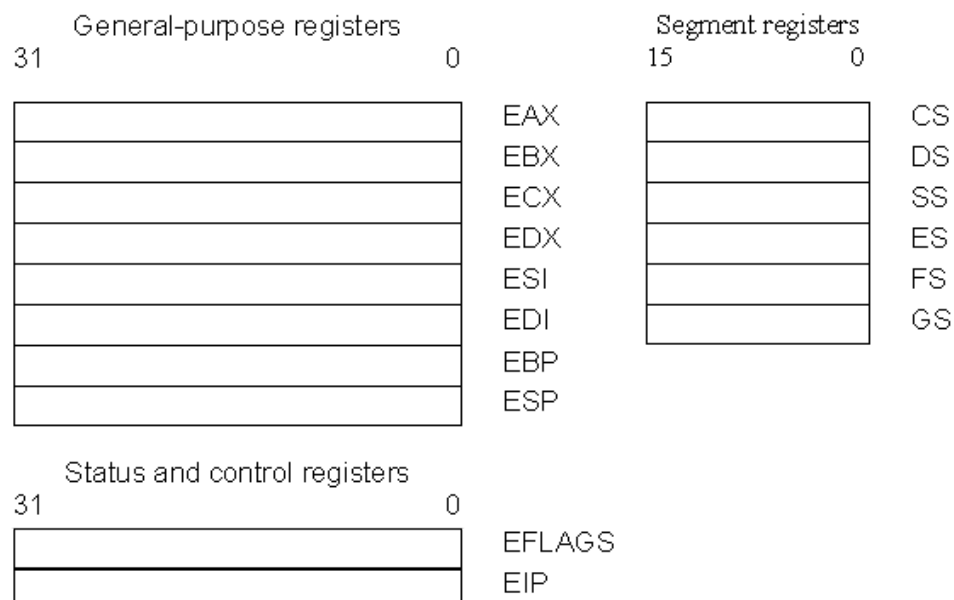
The IA32 processor has three operating modes:

- **Real-address mode.** This mode lets the processor to address "real" memory address. It can address up to 1Mbytes of memory (20-bit of address). It can also be called "unprotected" mode since operating system (such as DOS) code runs in the same mode as the user applications. IA32 processors have this mode to be compatible with early Intel processors such as 8086. The processor is set to this mode following by a power-up or a reset and can be switched to protected mode using a single instruction.
- **Protected mode.** This is the preferred mode for a modern operating system. It allows applications to use virtual memory addressing and supports multiple programming environment and protections.
- **System management mode.** This mode is designed for fast state snapshot and resumption. It is useful for power management

There is also a virtual-8086 mode that allows the processor to execute 8086 code software in the protected, multi-tasking environment.

1.2 Register Set

There are three types of registers: general-purpose data registers, segment registers, and status and control registers. The following figure shows these registers:



General-purpose Registers

The eight 32-bit general-purpose data registers are used to hold operands for logical and arithmetic operations, operands for address calculations and memory pointers. The following shows what they are used for:

- EAX—Accumulator for operands and results data.
- EBX—Pointer to data in the DS segment.
- ECX—Counter for string and loop operations.
- EDX—I/O pointer.
- ESI—Pointer to data in the segment pointed to by the DS register; source pointer for string operations.
- EDI—Pointer to data (or destination) in the segment pointed to by the ES register; destination pointer for string operations.
- EBP—Pointer to data on the stack (in the SS segment).
- ESP—Stack pointer (in the SS segment).

The following figure shows the lower 16 bits of the general-purpose registers can be used with the names AX, BX, CX, DX, BP, SP, SI, and DI (the names for the corresponding 32-bit ones have a prefix "E" for "extended"). Each of the lower two bytes of the EAX, EBX, ECX, and EDX registers can be referenced by the names AH, BH, CH, and DH (high bytes) and AL, BL, CL, and DL (low bytes).

General-purpose registers			16-bit	32-bit
31	16 15	8 7	0	
	AH	AL	AX	EAX
	BH	BL	BX	EBX
	CH	CL	CX	ECX
	DH	DL	DX	EDX
	BP			ESI
	SI			EDI
	DI			EBP
	SP			ESP

Segment Registers

There are six segment registers that hold 16-bit segment selectors. A segment selector is a special pointer that identifies a segment in memory. The six segment registers are:

- CS: code segment register
- SS: stack segment register
- DS, ES, FS, GS: data segment registers

Four data segment registers provide programs with flexible and efficient ways to access data.

Modern operating system and applications use the (unsegmented) memory model— all the segment registers are loaded with the same segment selector so that all memory references a program makes are to a single linear-address space.

When writing application code, you generally create segment selectors with assembler directives and symbols. The assembler and/or linker then creates the actual segment selectors associated with these directives and symbols. If you are writing system code, you may need to create segment selectors directly. (A detailed description of the segment-selector data structure is given in Chapter 3, Protected-Mode Memory Management, of the *IA32 Intel Architecture Software Developer's Manual, Volume 3*).

Project 1, 2, 3, and 4 all use the real-address mode and needs to set up the segment registers properly. Project 5 and 6 will use unsegmented memory model.

EFLAGS Register

The 32-bit EFLAGS register contains a group of status flags, a control flag, and a group of system flags. The following shows the function of EFLAGS register bits:

Function	EFLAG Register bit or bits
ID Flag (ID)	21 (system)
Virtual Interrupt Pending (VIP)	20 (system)
Virtual Interrupt Flag (VIF)	19 (system)
Alignment check (AC)	18 (system)
Virtual 8086 Mode (VM)	17 (system)
Resume Flag (RF)	16 (system)
Nested Task (NT)	14 (system)
I/O Privilege Level (IOPL)	13 to 12 (system)
Overflow Flag (OF)	11 (system)
Direction Flag (DF)	10 (system)
Interrupt Enable Flag (IF)	9 (system)
Trap Flag (TF)	8 (system)
Sign Flag (SF)	7 (status)
Zero Flag (ZF)	6 (status)
Auxiliary Carry Flag (AF)	4 (status)
Parity Flag (PF)	2 (status)
Carry Flag (CF)	0 (status)

Bits 1, 3, 5, 15, and 22 through 31 of this register are reserved. To understand what these fields mean and how to use them, please see Section 3.6.3 and 3.6.4 in *IA32 Intel Architecture Software Developer's Manual, Volume 1*.

EIP Register (Instruction Pointer)

The EIP register (or instruction pointer) can also be called "program counter." It contains the offset in the current code segment for the next instruction to be executed. It is advanced from one instruction boundary to the next in straight-line code or it is moved ahead or backwards by a number of instructions when executing JMP, Jcc, CALL, RET, and IRET instructions. The EIP cannot be accessed directly by software; it is controlled implicitly by control-transfer instructions (such as JMP, Jcc, CALL, and RET), inter-rupts, and exceptions. The EIP register can be loaded indirectly by modifying the value of a return instruction pointer on the procedure stack and executing a return instruction (RET or IRET).

Note that the value of the EIP may not match with the current instruction because of instruction prefetching. The only way to read the EIP is to execute a CALL instruction and then read the value of the return instruction pointer from the procedure stack.

The IA32 processors also have control registers, which can be found in the Intel/manuals.

1.3 Addressing

Bit and Byte Order

IA32 processors use "little endian" as their byte order. This means that the bytes of a word are numbered starting from the least significant byte and that the least significant bit starts of a word starts in the least significant byte.

Data Types

IA32 provides four data types: a byte (8 bits), a word (16 bits), a double-word (32 bits), and a quad-word (64 bits). Note that a word is "word" in Gnu assembler and a double-word is equivalent to "long" in Gnu assembler.

Memory Addressing

One can use either flat memory model or segmented memory mode. With the flat memory model, memory appears to a program as a single, continuous address space, called a linear address space. Code (a program's instructions), data, and the procedure stack are all contained in this address space. The linear address space is byte addressable, with addresses running contiguously from 0 to $2^{32} - 1$.

With the segmented memory mode, memory appears to a program as a group of independent address spaces called segments. When using this model, code, data, and stacks are typically contained in separate segments. To address a byte in a segment, a program must issue a logical address, which consists of a segment selector and an offset. (A logical address is often referred to as a far pointer.) The segment selector identifies the segment to be accessed and the offset identifies a byte in the address space of the segment. The programs running on an IA32 processor can address up to 16,383 segments of different sizes and types. Internally, all the segments that are defined for a system are mapped into the processor's linear address space. So, the processor translates each logical address into a linear address to access a memory location. This translation is transparent to the application program.

1.4 Processor Reset

A cold boot or a warm boot can reset the CPU. A cold boot is powering up a system whereas a warm boot means that when three keys CTRL-ALT-DEL are all pressed together, the keyboard BIOS will set a special flag and resets the CPU.

Upon reset, the processor sets itself to real-mode with interrupts disabled and key registers set to a known state. For example, the state of the EFLAGS register is 00000002H and the memory is unchanged. Thus, the memory will contain garbage upon a cold boot. The CPU will jump to the BIOS (Basic Input Output Services) to load the bootstrap loader program from the diskette drive or the hard disk and begins execution of the loader. The BIOS loads the bootstrap loader into the fixed address 0:7C00 and jumps to the starting address.

2 Assembly Programming

It often takes a while to master the techniques to program in assembly language for a particular machine. On the other hand, it should not take much time to assembly programming on IA32 processors if you are familiar with assembly programming for another processor. This section assumes that you are already familiar with Gnu assembly syntax (learned from the course *Introduction to Programming Systems* or its equivalent).

2.1 Instruction Syntax

There are two conventions about their syntax and representations: Intel and AT&T. Most documents use the Intel convention, whereas the Gnu assembler uses the AT&T convention. The main differences are:

	Gnu Syntax (AT&T)	Intel
Immediate operands	Preceded by "\$" e.g.: push \$4 movl \$0xd00a, %eax	Undelimited e.g.: push 4 mov ebx, d00ah
Register operands	Preceded by "%" e.g.: %eax	Undelimited e.g.: eax
Argument order (e.g. adds the address of C variable "foo" to register EAX)	source1, [source2,] dest e.g.: addl \$_foo, %eax	dest, source1 [, source2] e.g.: add eax, _foo

Single-size operands	Explicit with operand sizes opcode{b,w,l} e.g.: <code>movb foo, %al</code>	Implicit with register name, byte ptr , word ptr , or dword ptr e.g.: <code>mov al, foo</code>
Address a C variable "foo"	<code>_foo</code>	<code>[_foo]</code>
Address memory pointed by a register (e.g. EAX)	<code>(%eax)</code>	<code>[eax]</code>
Address a variable offset by a value in the register	<code>_foo(%eax)</code>	<code>[eax + _foo]</code>
Address a value in an array "foo" of 32-bit integers	<code>_foo(, %eax, 4)</code>	<code>[eax*4+foo]</code>
Equivalent to C code <code>*(p+1)</code>	<code>1(%eax)</code>	If EAX holds the value of p, then <code>[eax+1]</code>

2.2 Memory operands

IA32 processors use segmented memory architecture. It means that the memory locations are referenced by means of a segment selector and an offset:

- The segment selector specifies the segment containing the operand, and
- The offset (the number of bytes from the beginning of the segment to the first byte of the operand) specifies the linear or effective address of the operand.

The segment selector can be specified either implicitly or explicitly. The most common method of specifying a segment selector is to load it in a segment register and then allow the processor to select the register implicitly, depending on the type of operation being performed. The processor automatically chooses a segment according to the following rules:

- Code segment register CS for instruction fetches
- Stack segment register SS for stack pushes and pops as well as references using ESP or EBP as a base register
- Data segment register DS for all data references except when relative to stack or string destination
- Data segment register ES for the destinations of string instructions

The offset part of the memory address can be specified either directly as a static value (called a *displacement*) or through an address computation made up of one or more of the following components:

- Displacement—An 8-, 16-, or 32-bit value.
- Base—The value in a general-purpose register.
- Index—The value in a general-purpose register except EBP.
- Scale factor—A value of 2, 4, or 8 that is multiplied by the index value.

An effective address is computed by:

$$\text{Offset} = \text{Base} + (\text{Index} \times \text{Scale}) + \text{displacement}$$

The offset which results from adding these components is called an *effective address* of the selected segment. Each of these components can have either a positive or negative (2's complement) value, with the exception of the scaling factor.

2.3. Frequently Used Instructions

The following is a small set of frequently used instructions:

Category	Instructions	Explanations
Data Transfer	<code>mov{l,w,b} source, dest</code>	Move from source to dest
	<code>xchg{l,w,b} dest1, dest2</code>	Exchange
	<code>cmpxchg{l,w,b} dest1, dest2</code>	Compare and exchange
	<code>push/pop{l,w}</code>	Push onto / pop off the stack
	<code>movsb</code>	Move bytes at DS:(E)SI to address ES:(E)DI, typically prefix with <code>rep</code>
Arithmetic	<code>add/sub{l,w,b} source, dest</code>	Add/subtract
	<code>imul/mul{l,w,b} formats</code>	Signed/unsigned multiply
	<code>idiv/div{l,w,b} dest</code>	Signed/unsigned divide

Logic	inc/dec/neg{l,w,b} dest	Increment/decrement/negate
	cmp{l,w,b} source1, source2	Compare
	and/or/xor/not{l,w,b} source, dest	Logic and/or/xor/not operation
	sal/sar{l,w,b} formats	Arithmetic shift left/right
	shl/shr{l,w,b} formats	Logic shift left/right
Control transfer	jmp address	Unconditional jump
	call address	Save EIP on the stack jump to address
	ret	Return to the EIP location saved by call
	leave	Restore EBP from the stack; pop off the stack frame
	j{e,ne,l,le,g,ge} address	Jump to address if {=,!=,<,<=,>,>=}
	loop address	Decrement ECX or CX; jump if = 0
	rep	Repeat string operation prefix
	int number	Software interrupt
	iret	Return from interrupt; pop EFLAGS from the stack

In addition, the name for a long JUMP is `ljmp` and long CALL is `lcall`.

This is again a small set of instructions. Section 3.2 of *IA32 Intel Architecture Software Developer's Manual, Volume 2* provides a complete set of the IA32 instructions and the detailed description for each instruction. The instruction names in the Intel manual uses the Intel convention (obviously) and you need to convert them to the AT&T syntax.

2.4 Assembler Directives

The Gnu assembler directive names begin with a period "." and the rest are letters in lower case. Here are some examples of commonly used directives:

```
.ascii "string foo" defines an ASCII string "string foo"
.asciz "string foo" defines an ASCII string "string foo" with a zero at the end
.string "string foo" is the same as .asciz "string foo"
.align 4 aligns the memory at double-word boundary
.byte 10, 13, 0 defines three bytes
.word 0x0456, 0x1234 defines two words
.long 0x001234, 0x12345 defines two long words
.equ STACK_SEGMENT, 0x9000 sets symbol STACK_SEGMENT the value 0x9000
.globl symbol makes "symbol" global (useful for defining global labels and procedure names)
.code16 tells the assembler to insert the appropriate override prefixes so the code will run in real mode.
```

When using directives to define a string, bytes or a word, you often want to make sure that they are aligned to 32-bit long word by padding additional bytes.

2.5 Inline Assembly

The most basic format of inline assembly code into your the assembly code generated by the gcc compiler is to use

```
asm volatile ( "assembly-instruction" );
```

where `assembly-instruction` will be inlined into where the `asm` statement is. The key word `volatile` is optional. It tells the gcc compiler not to optimize this instruction away. This is a very convenient way to inline assembly instructions that require no registers. For example, you can use

```
asm volatile( "cli" );
```

to clear interrupts and

```
asm volatile( "sti" );
```

to enable interrupts.

The general format to write inline assembly code in C is:

```
asm [volatile]( "statements": output_regs: input_regs: used_regs);
```

where `statements` are the assembly instructions. If there are more than one instruction, you can use `"\n\t"` to separate them to make them look pretty. `"input_regs"` tells gcc compiler which C variables move to which registers. For example, if you would like to load variable `"foo"` into register `EAX` and `"bar"` into register `ECX`, you would say

```
: "a" (foo), "c" (bar)
```

gcc uses single letters to represent all registers:

Single Letters	Registers
a	eax
b	ebx
c	ecx
d	edx
S	esi
D	edi
I	constant value (0 to 31)
q	allocate a register from EAX, EBX, ECX, EDX
r	allocate a register from EAX, EBX, ECX, EDX, ESI, EDI

Note that you cannot specify register `AH` or `AL` this way. You need to get to `EAX` first and then go from there.

`"output_regs"` provides output registers. A convenient way to do this is to let gcc compiler to pick the registers for you. You need to say `"=q"` or `"=r"` to let gcc compiler pick registers for you. You can refer to the first allocated register with `"%0"`, second with `"%1"`, and so on, in the assembly instructions. If you refer to the registers in the input register list, you simply say `"0"` or `"1"` without the `"%"` prefix.

`"used_regs"` lists the registers that are used (or clobbered) in the assembly code.

To understand exactly how to do this, please try to use gcc to compile a piece of C code containing the following inline assembly:

```
asm ("leal (%1,%1,4), %0"
    : "=r" (x_times_5)
    : "r" (x) );
```

and

```
asm ("leal (%0,%0,4), %0"
    : "=r" (x)
    : "0" (x) );
```

2.6 Program Structure and Calling Conventions

The simplest way to learn assembly programming is to compile a simple C program into its assembly source code as a template. The source code will tell you common opcodes, directives and addressing syntax. This is an efficient way to learn assembly programming.

The following is an example to show the program structure and calling conventions. Consider the following C program `hello.c`:

```
#include <stdio.h>

static char buf[ 4096 ];

int foo( int n )
{
```

```

    return n - 1;
}

int main (void)
{
    printf( "Hello world\n" );
    return foo( 5);
}

```

Issue the command on a shell:

```
gcc -S hello.c
```

gcc compiler will compile hello.c into its assembly source file in the same directory called hello.s. After reading this document, you should find the assembly code self-explanatory. In case you have questions, following provides some comments on the instructions related to calling conventions.

```

    .file "hello.c"
    .text
.globl foo                                # "foo" is a global name
    .type foo,@function                  # "foo" is a function type
foo:
    pushl %ebp                          # push ebp onto stack
    movl %esp, %ebp                     # move stack pointer to ebp
    movl 8(%ebp), %eax
    decl %eax
    leave                                # restore esp and ebp
    ret                                 # return to caller
.Lfe1:
    .size foo,.Lfe1-foo
    .section .rodata
.LC0:
    .string "Hello world\n"
    .text
.globl main
    .type main,@function
main:
    pushl %ebp                          # push ebp onto stack
    movl %esp, %ebp                     # move stack pointer esp to ebp
    subl $8, %esp
    andl $-16, %esp
    movl $0, %eax
    subl %eax, %esp
    subl $12, %esp
    pushl $.LC0
    call printf
    addl $16, %esp
    subl $12, %esp
    pushl $5                            # push arg to stack
    call foo                            # call foo function
    addl $16, %esp
    leave                                # restore esp and ebp
    ret                                 # return
.Lfe2:
    .size main,.Lfe2-main
    .local buf
    .comm buf,4096,32
    .ident "GCC: (GNU) 3.2.2 20030222 (Red Hat Linux 3.2.2-5)"

```

You should try to create a few program examples in C and use gcc to compile them into assembly as case studies.

3 BIOS Services

The book *Undocumented PC* provides detailed BIOS (Basic Input/Output System) services. This document presents a very small set of services used in our course projects.

3.1 Display Memory

PC's display RAM is mapped into memory space. One can write directly to the screen by writing to the display RAM starting at 0xb800:0000. Each location on the screen requires two bytes---one to specify the attribute (Use **0x07** for white color) and the second for the character itself. The text screen has 25 lines and 80 characters per line. So, to write to i-th row and j-th column, you write the 2 bytes starting at offset $((i-1)*80+(j-1))*2$.

So, the following code sequence writes the character 'K' (ascii 0x4b) to the top left corner of the screen.

```
movw 0xb800,%bx
movw %bx,%es
movw $0x074b,%es:(0x0)
```

This code sequence is useful for debugging programs during booting.

3.2 Write to Display at Current Cursor

To send a character to the display at the current cursor position on the active display, one can use the BIOS service:

int 0x10

with the following parameters

- ah = 0x0e, indicating this is function 0x0e
- al = holding the character to write
- bh = active page number (Use 0x00)
- bl = foreground color (graphics mode only) (Use 0x02)

The service returns the character displayed. Note that the linefeed character is 0x0a and carriage return is 0x0d.

This service call automatically wraps lines, scrolls and interprets some control characters for specific actions.

3.3 Read from Diskette

The BIOS service call for reading 512-byte diskette sectors from a specified location uses the software interrupt

int 0x13

with the following parameters set up:

- ah = 2, to indicate this is function 2
- al = number of sectors to read, 1 to 36
- ch = track number, 0 to 79
- cl = sector number, 1 to 36
- dh = head number, 0 or 1
- dl = drive number, 0 to 3
- es:bx = pointer where to place information read from diskette

This service call will return the following:

- ah = return status (0 if successful)
- al = number of sectors read
- carry = 0 successful, = 1 if error occurred

The data read is placed into RAM at the location specified by ES:BX. The buffer must be sufficiently large to hold the data and must not cross a 64K linear address boundary.